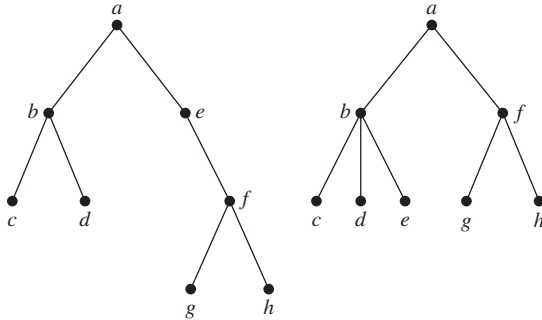
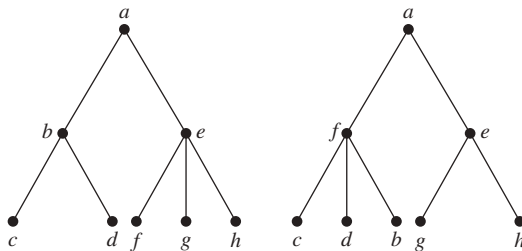


- \*27. Show that an ordered rooted tree is uniquely determined when a list of vertices generated by a postorder traversal of the tree and the number of children of each vertex are specified.
28. Show that preorder traversals of the two ordered rooted trees displayed below produce the same list of vertices. Note that this does not contradict the statement in Exercise 26, because the numbers of children of internal vertices in the two ordered rooted trees differ.



29. Show that postorder traversals of these two ordered rooted trees produce the same list of vertices. Note that this does not contradict the statement in Exercise 27, because the numbers of children of internal vertices in the two ordered rooted trees differ.



**Well-formed formulae** in prefix notation over a set of symbols and a set of binary operators are defined recursively by these rules:

- (i) if  $x$  is a symbol, then  $x$  is a well-formed formula in prefix notation;
- (ii) if  $X$  and  $Y$  are well-formed formulae and  $*$  is an operator, then  $*XY$  is a well-formed formula.

30. Which of these are well-formed formulae over the symbols  $\{x, y, z\}$  and the set of binary operators  $\{\times, +, \circ\}$ ?

- a)  $\times + + xyx$
- b)  $\circ xy \times xz$
- c)  $\times \circ xz \times \times xy$
- d)  $\times + \circ x x \circ x x x$

- \*31. Show that any well-formed formula in prefix notation over a set of symbols and a set of binary operators contains exactly one more symbol than the number of operators.
32. Give a definition of well-formed formulae in postfix notation over a set of symbols and a set of binary operators.
33. Give six examples of well-formed formulae with three or more operators in postfix notation over the set of symbols  $\{x, y, z\}$  and the set of operators  $\{+, \times, \circ\}$ .
34. Extend the definition of well-formed formulae in prefix notation to sets of symbols and operators where the operators may not be binary.

## 11.4 Spanning Trees

### 11.4.1 Introduction

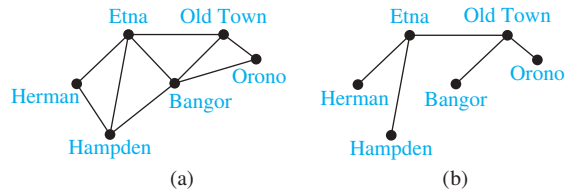
Consider the system of roads in Maine represented by the simple graph shown in Figure 1(a). The only way the roads can be kept open in the winter is by frequently plowing them. The highway department wants to plow the fewest roads so that there will always be cleared roads connecting any two towns. How can this be done?

At least five roads must be plowed to ensure that there is a path between any two towns. Figure 1(b) shows one such set of roads. Note that the subgraph representing these roads is a tree, because it is connected and contains six vertices and five edges.

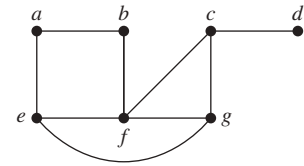
This problem was solved with a connected subgraph with the minimum number of edges containing all vertices of the original simple graph. Such a graph must be a tree.

#### Definition 1

Let  $G$  be a simple graph. A *spanning tree* of  $G$  is a subgraph of  $G$  that is a tree containing every vertex of  $G$ .



**FIGURE 1** (a) A road system and (b) a set of roads to plow.

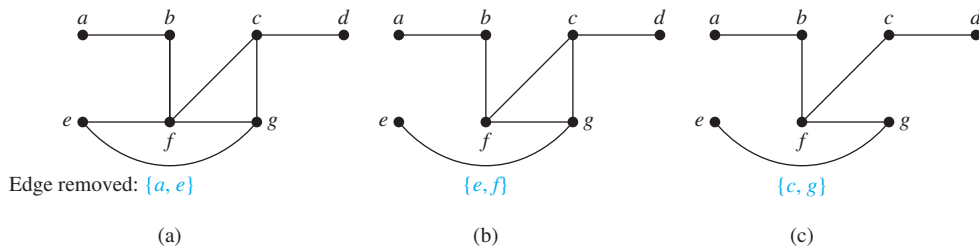


**FIGURE 2** The simple graph  $G$ .

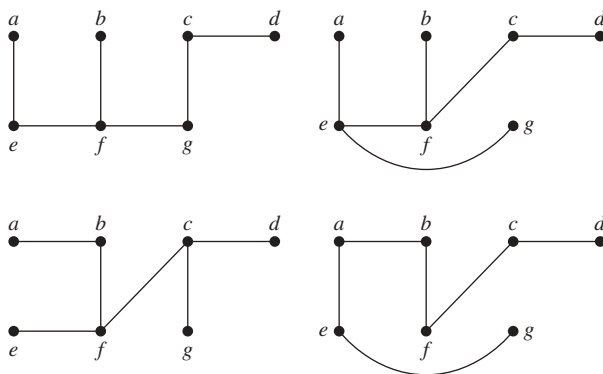
A simple graph with a spanning tree must be connected, because there is a path in the spanning tree between any two vertices. The converse is also true; that is, every connected simple graph has a spanning tree. We will give an example before proving this result.

**EXAMPLE 1** Find a spanning tree of the simple graph  $G$  shown in Figure 2.

**Solution:** The graph  $G$  is connected, but it is not a tree because it contains simple circuits. Remove the edge  $\{a, e\}$ . This eliminates one simple circuit, and the resulting subgraph is still connected and still contains every vertex of  $G$ . Next remove the edge  $\{e, f\}$  to eliminate a second simple circuit. Finally, remove edge  $\{c, g\}$  to produce a simple graph with no simple circuits. This subgraph is a spanning tree, because it is a tree that contains every vertex of  $G$ . The sequence of edge removals used to produce the spanning tree is illustrated in Figure 3.



**FIGURE 3** Producing a spanning tree for  $G$  by removing edges that form simple circuits.



**FIGURE 4** Spanning trees of  $G$ .

The tree shown in Figure 3 is not the only spanning tree of  $G$ . For instance, each of the trees shown in Figure 4 is a spanning tree of  $G$ . ▶

### THEOREM 1

A simple graph is connected if and only if it has a spanning tree.

**Proof:** First, suppose that a simple graph  $G$  has a spanning tree  $T$ .  $T$  contains every vertex of  $G$ . Furthermore, there is a path in  $T$  between any two of its vertices. Because  $T$  is a subgraph of  $G$ , there is a path in  $G$  between any two of its vertices. Hence,  $G$  is connected.

Now suppose that  $G$  is connected. If  $G$  is not a tree, it must contain a simple circuit. Remove an edge from one of these simple circuits. The resulting subgraph has one fewer edge but still contains all the vertices of  $G$  and is connected. This subgraph is still connected because when two vertices are connected by a path containing the removed edge, they are connected by a path not containing this edge. We can construct such a path by inserting into the original path, at the point where the removed edge once was, the simple circuit with this edge removed. If this subgraph is not a tree, it has a simple circuit; so as before, remove an edge that is in a simple circuit. Repeat this process until no simple circuits remain. This is possible because there are only a finite number of edges in the graph. The process terminates when no simple circuits remain. A tree is produced because the graph stays connected as edges are removed. This tree is a spanning tree because it contains every vertex of  $G$ . ◀

Spanning trees are important in data networking, as Example 2 shows.

### EXAMPLE 2

Links ▶

**IP Multicasting** Spanning trees play an important role in multicasting over Internet Protocol (IP) networks. To send data from a source computer to multiple receiving computers, each of which is a subnetwork, data could be sent separately to each computer. This type of networking, called unicasting, is inefficient, because many copies of the same data are transmitted over the network. To make the transmission of data to multiple receiving computers more efficient, IP multicasting is used. With IP multicasting, a computer sends a single copy of data over the network, and as data reaches intermediate routers, the data are forwarded to one or more other routers so that ultimately all receiving computers in their various subnetworks receive these data. (Routers are computers that are dedicated to forwarding IP datagrams between subnetworks in a network. In multicasting, routers use Class D addresses, each representing a session that receiving computers may join; see Example 17 in Section 6.1.)

For data to reach receiving computers as quickly as possible, there should be no loops (which in graph theory terminology are circuits or cycles) in the path that data take through the network. That is, once data have reached a particular router, data should never return to this router. To avoid loops, the multicast routers use network algorithms to construct a spanning tree in the graph that has the multicast source, the routers, and the subnetworks containing receiving computers as vertices, with edges representing the links between computers and/or routers. The root of this spanning tree is the multicast source. The subnetworks containing receiving computers are leaves of the tree. (Note that subnetworks not containing receiving stations are not included in the graph.) This is illustrated in Figure 5. ◀

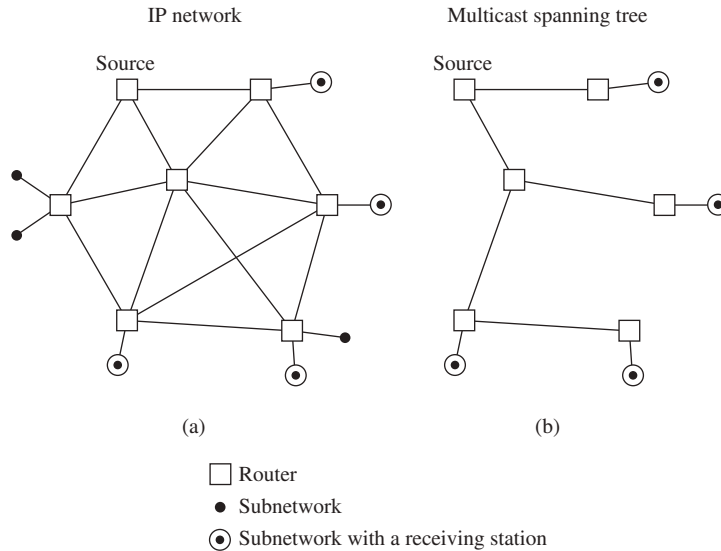
## 11.4.2 Depth-First Search

Links ▶

The proof of Theorem 1 gives an algorithm for finding spanning trees by removing edges from simple circuits. This algorithm is inefficient, because it requires that simple circuits be identified. Instead of constructing spanning trees by removing edges, spanning trees can be built up by successively adding edges. Two algorithms based on this principle will be presented here.

Demo ▶

We can build a spanning tree for a connected simple graph using **depth-first search**. We will form a rooted tree, and the spanning tree will be the underlying undirected graph of this rooted tree. Arbitrarily choose a vertex of the graph as the root. Form a path starting at this vertex by successively adding vertices and edges, where each new edge is incident with the last vertex in the path and a vertex not already in the path. Continue adding vertices and edges to this path as long as possible. If the path goes through all vertices of the graph, the tree consisting of this path is a spanning tree. However, if the path does not go through all vertices, more vertices and edges must be added. Move back to the next to last vertex in the path, and, if possible, form a new path



**FIGURE 5** A multicast spanning tree.

starting at this vertex passing through vertices that were not already visited. If this cannot be done, move back another vertex in the path, that is, two vertices back in the path, and try again.

Repeat this procedure, beginning at the last vertex visited, moving back up the path one vertex at a time, forming new paths that are as long as possible until no more edges can be added. Because the graph has a finite number of edges and is connected, this process ends with the production of a spanning tree. Each vertex that ends a path at a stage of the algorithm will be a leaf in the rooted tree, and each vertex where a path is constructed starting at this vertex will be an internal vertex.

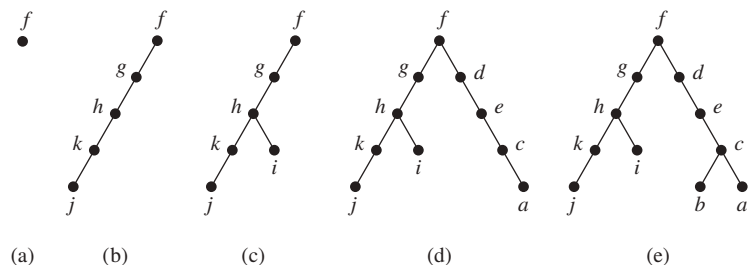
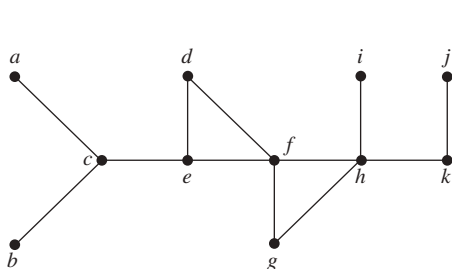
The reader should note the recursive nature of this procedure. Also, note that if the vertices in the graph are ordered, the choices of edges at each stage of the procedure are all determined when we always choose the first vertex in the ordering that is available. However, we will not always explicitly order the vertices of a graph.

Depth-first search is also called **backtracking**, because the algorithm returns to vertices previously visited to add paths. Example 3 illustrates backtracking.

**EXAMPLE 3** Use depth-first search to find a spanning tree for the graph  $G$  shown in Figure 6.

**Extra Examples**

**Solution:** The steps used by depth-first search to produce a spanning tree of  $G$  are shown in Figure 7. We arbitrarily start with the vertex  $f$ . A path is built by successively adding edges incident with vertices not already in the path, as long as this is possible. This produces a path  $f, g, h, k, j$  (note that other paths could have been built). Next, backtrack to  $k$ . There is no path beginning at  $k$  containing vertices not already visited. So we backtrack to  $h$ . Form the



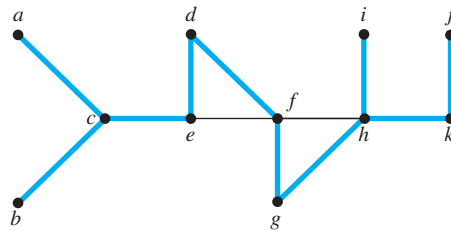
**FIGURE 6** The graph  $G$ .

**FIGURE 7** Depth-first search of  $G$ .

path  $h, i$ . Then backtrack to  $h$ , and then to  $f$ . From  $f$  build the path  $f, d, e, c, a$ . Then backtrack to  $c$  and form the path  $c, b$ . This produces the spanning tree. ◀

The edges selected by depth-first search of a graph are called **tree edges**. All other edges of the graph must connect a vertex to an ancestor or descendant of this vertex in the tree. These edges are called **back edges**. (Exercise 43 asks for a proof of this fact.)

**EXAMPLE 4** In Figure 8 we highlight the tree edges found by depth-first search starting at vertex  $f$  by showing them with heavy colored lines. The back edges  $(e, f)$  and  $(f, h)$  are shown with thinner black lines. ◀



**FIGURE 8** The tree edges and back edges of the depth-first search in example 4.

We have explained how to find a spanning tree of a graph using depth-first search. However, our discussion so far has not brought out the recursive nature of depth-first search. To help make the recursive nature of the algorithm clear, we need a little terminology. We say that we *explore* from a vertex  $v$  when we carry out the steps of depth-first search beginning when  $v$  is added to the tree and ending when we have backtracked back to  $v$  for the last time. The key observation needed to understand the recursive nature of the algorithm is that when we add an edge connecting a vertex  $v$  to a vertex  $w$ , we finish exploring from  $w$  before we return to  $v$  to complete exploring from  $v$ .

In Algorithm 1 we construct the spanning tree of a graph  $G$  with vertices  $v_1, \dots, v_n$  by first selecting the vertex  $v_1$  to be the root. We initially set  $T$  to be the tree with just this one vertex. At each step we add a new vertex to the tree  $T$  together with an edge from a vertex already in  $T$  to this new vertex and we explore from this new vertex. Note that at the completion of the algorithm,  $T$  contains no simple circuits because no edge is ever added that connects two vertices in the tree. Moreover,  $T$  remains connected as it is built. (These last two observations can be easily proved via mathematical induction.) Because  $G$  is connected, every vertex in  $G$  is visited by the algorithm and is added to the tree (as the reader should verify). It follows that  $T$  is a spanning tree of  $G$ .

#### ALGORITHM 1 Depth-First Search.

**procedure**  $DFS(G$ : connected graph with vertices  $v_1, v_2, \dots, v_n$ )  
 $T :=$  tree consisting only of the vertex  $v_1$   
 $visit(v_1)$

**procedure**  $visit(v$ : vertex of  $G$ )  
**for** each vertex  $w$  adjacent to  $v$  and not yet in  $T$   
    add vertex  $w$  and edge  $\{v, w\}$  to  $T$   
     $visit(w)$

We now analyze the computational complexity of the depth-first search algorithm. The key observation is that for each vertex  $v$ , the procedure  $visit(v)$  is called when the vertex  $v$  is first encountered in the search and it is not called again. Assuming that the adjacency lists for  $G$  are available (see Section 10.3), no computations are required to find the vertices adjacent to  $v$ . As we follow the steps of the algorithm, we examine each edge at most twice to determine whether to add this edge and one of its endpoints to the tree. Consequently, the procedure  $DFS$  constructs a spanning tree using  $O(e)$ , or  $O(n^2)$ , steps where  $e$  and  $n$  are the number of edges and vertices in  $G$ , respectively. [Note that a step involves examining a vertex to see whether it is already in the spanning tree as it is being built and adding this vertex and the corresponding edge if the vertex is not already in the tree. We have also made use of the inequality  $e \leq n(n-1)/2$ , which holds for any simple graph.]

Depth-first search can be used as the basis for algorithms that solve many different problems. For example, it can be used to find paths and circuits in a graph, it can be used to determine the connected components of a graph, and it can be used to find the cut vertices of a connected graph. As we will see, depth-first search is the basis of backtracking techniques used to search for solutions of computationally difficult problems. (See [GrYe05], [Ma89], and [CoLeRiSt09] for a discussion of algorithms based on depth-first search.)

### 11.4.3 Breadth-First Search

Demo

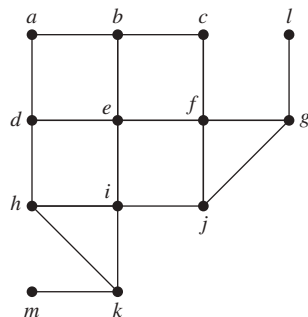
Links

We can also produce a spanning tree of a simple graph by the use of **breadth-first search**. Again, a rooted tree will be constructed, and the underlying undirected graph of this rooted tree forms the spanning tree. Arbitrarily choose a root from the vertices of the graph. Then add all edges incident to this vertex. The new vertices added at this stage become the vertices at level 1 in the spanning tree. Arbitrarily order them. Next, for each vertex at level 1, visited in order, add each edge incident to this vertex to the tree as long as it does not produce a simple circuit. Arbitrarily order the children of each vertex at level 1. This produces the vertices at level 2 in the tree. Follow the same procedure until all the vertices in the tree have been added. The procedure ends because there are only a finite number of edges in the graph. A spanning tree is produced because we have produced a tree containing every vertex of the graph. An example of breadth-first search is given in Example 5.

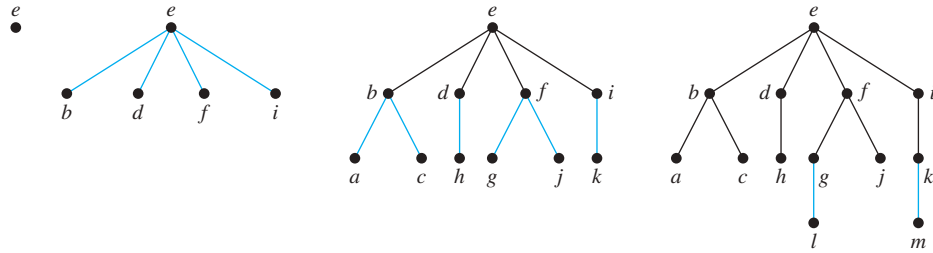
**EXAMPLE 5** Use breadth-first search to find a spanning tree for the graph shown in Figure 9.

Extra Examples

**Solution:** The steps of the breadth-first search procedure are shown in Figure 10. We choose the vertex  $e$  to be the root. Then we add edges incident with all vertices adjacent to  $e$ , so edges from  $e$  to  $b, d, f$ , and  $i$  are added. These four vertices are at level 1 in the tree. Next, add the edges from these vertices at level 1 to adjacent vertices not already in the tree. Hence, the edges from  $b$  to  $a$  and  $c$  are added, as are edges from  $d$  to  $h$ , from  $f$  to  $j$  and  $g$ , and from  $i$  to  $k$ . The new



**FIGURE 9** A graph  $G$ .



**FIGURE 10** Breadth-first search of  $G$ .

vertices  $a, c, h, j, g$ , and  $k$  are at level 2. Next, add edges from these vertices to adjacent vertices not already in the graph. This adds edges from  $g$  to  $l$  and from  $k$  to  $m$ . ◀

We describe breadth-first search in pseudocode as Algorithm 2. In this algorithm, we assume the vertices of the connected graph  $G$  are ordered as  $v_1, v_2, \dots, v_n$ . In the algorithm we use the term “process” to describe the procedure of adding new vertices, and corresponding edges, to the tree adjacent to the current vertex being processed as long as a simple circuit is not produced.

**ALGORITHM 2** Breadth-First Search.

```

procedure  $BFS(G: \text{connected graph with vertices } v_1, v_2, \dots, v_n)$ 
 $T :=$  tree consisting only of vertex  $v_1$ 
 $L :=$  empty list
put  $v_1$  in the list  $L$  of unprocessed vertices
while  $L$  is not empty
    remove the first vertex,  $v$ , from  $L$ 
    for each neighbor  $w$  of  $v$ 
        if  $w$  is not in  $L$  and not in  $T$  then
            add  $w$  to the end of the list  $L$ 
            add  $w$  and edge  $\{v, w\}$  to  $T$ 
  
```

We now analyze the computational complexity of breadth-first search. For each vertex  $v$  in the graph we examine all vertices adjacent to  $v$  and we add each vertex not yet visited to the tree  $T$ . Assuming we have the adjacency lists for the graph available, no computation is required to determine which vertices are adjacent to a given vertex. As in the analysis of the depth-first search algorithm, we see that we examine each edge at most twice to determine whether we should add this edge and its endpoint not already in the tree. It follows that the breadth-first search algorithm uses  $O(e)$  or  $O(n^2)$  steps.

Breadth-first search is one of the most useful algorithms in graph theory. In particular, it can serve as the basis for algorithms that solve a wide variety of problems. For example, algorithms that find the connected components of a graph, that determine whether a graph is bipartite, and that find the path with the fewest edges between two vertices in a graph can all be built using breadth-first search.

**Breadth-First Search versus Depth-First Search** We have introduced two widely used algorithms for constructing spanning trees of graphs, breadth-first search (BFS) and depth-first search (DFS). Both algorithms can be used to produce a spanning tree when given a connected graph. But why might it be better to use one rather than the other?

Although both BFS and DFS can be used to solve the same problems, there are theoretical and practical reasons for choosing between them. To solve a particular type of problem one of these search algorithms can be easier to apply, or can provide more insight, than the other. For some problems, BFS can be easier to use than DFS because BFS partitions the vertices of a graph into levels, which tell us how far away vertices are from the root. Also, edges connect vertices at the same level or at adjoining levels (see Exercise 34). On the other hand, there are many types of problems that are more naturally solved using DFS, such as those discussed in Examples 6–8. Generally, DFS is a better choice when we need to explore a graph by exploring deeper rather than by exploring the graph systematically level by level. The structure obtained when using DFS (see Exercise 51) can also be used when solving problems.

Both BFS and DFS are extensively used in practice. The choice for which to use often depends on implementation details, such as the data structures used. Time and space considerations are paramount, especially when the problems being solved involved huge graphs. Also, keep in mind that we often solve problems using graph searching without completing the task of finding a spanning tree. When we use BFS on a graph that is dense, we spend a lot of time and use a lot of space to explore the graph level by level. In such situations, it can be preferable to use DFS to quickly reach vertices far away from the root. For a sparse graph, however, it may be more efficient to explore a graph level by level.

### 11.4.4 Backtracking Applications

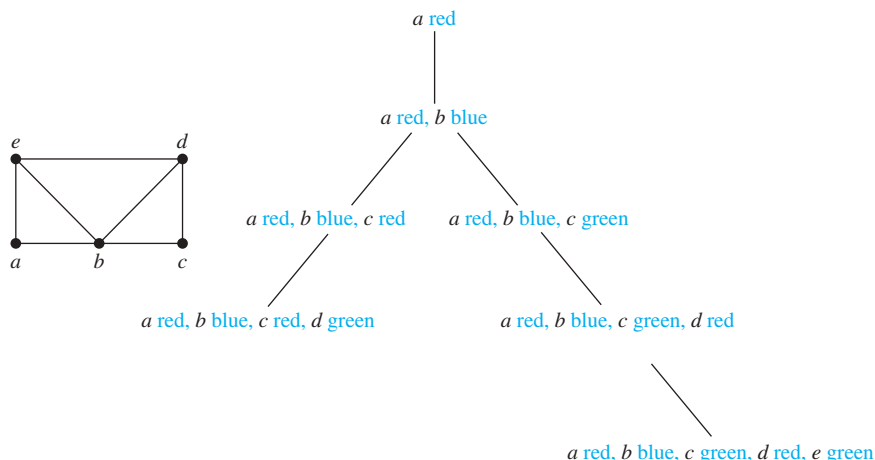
There are problems that can be solved only by performing an exhaustive search of all possible solutions. One way to search systematically for a solution is to use a decision tree, where each internal vertex represents a decision and each leaf a possible solution. To find a solution via backtracking, first make a sequence of decisions in an attempt to reach a solution as long as this is possible. The sequence of decisions can be represented by a path in the decision tree. Once it is known that no solution can result from any further sequence of decisions, backtrack to the parent of the current vertex and work toward a solution with another series of decisions, if this is possible. The procedure continues until a solution is found, or it is established that no solution exists. Examples 6 to 8 illustrate the usefulness of backtracking.

**EXAMPLE 6 Graph Colorings** How can backtracking be used to decide whether a graph can be colored using  $n$  colors?

**Solution:** We can solve this problem using backtracking in the following way. First pick some vertex  $a$  and assign it color 1. Then pick a second vertex  $b$ , and if  $b$  is not adjacent to  $a$ , assign it color 1. Otherwise, assign color 2 to  $b$ . Then go on to a third vertex  $c$ . Use color 1, if possible, for  $c$ . Otherwise use color 2, if this is possible. Only if neither color 1 nor color 2 can be used should color 3 be used. Continue this process as long as it is possible to assign one of the  $n$  colors to each additional vertex, always using the first allowable color in the list. If a vertex is reached that cannot be colored by any of the  $n$  colors, backtrack to the last assignment made and change the coloring of the last vertex colored, if possible, using the next allowable color in the list. If it is not possible to change this coloring, backtrack farther to previous assignments, one step back at a time, until it is possible to change a coloring of a vertex. Then continue assigning colors of additional vertices as long as possible. If a coloring using  $n$  colors exists, backtracking will produce it. (Unfortunately this procedure can be extremely inefficient.)

In particular, consider the problem of coloring the graph shown in Figure 11 with three colors. The tree shown in Figure 11 illustrates how backtracking can be used to construct a 3-coloring. In this procedure, red is used first, then blue, and finally green. This simple example can obviously be done without backtracking, but it is a good illustration of the technique.

In this tree, the initial path from the root, which represents the assignment of red to  $a$ , leads to a coloring with  $a$  red,  $b$  blue,  $c$  red, and  $d$  green. It is impossible to color  $e$  using any of the three colors when  $a$ ,  $b$ ,  $c$ , and  $d$  are colored in this way. So, backtrack to the parent of the



**FIGURE 11** Coloring a graph using backtracking.

vertex representing this coloring. Because no other color can be used for  $d$ , backtrack one more level. Then change the color of  $c$  to green. We obtain a coloring of the graph by then assigning red to  $d$  and green to  $e$ .

**EXAMPLE 7 The  $n$ -Queens Problem** The  $n$ -queens problem asks how  $n$  queens can be placed on an  $n \times n$  chessboard so that no two queens can attack one another. How can backtracking be used to solve the  $n$ -queens problem?

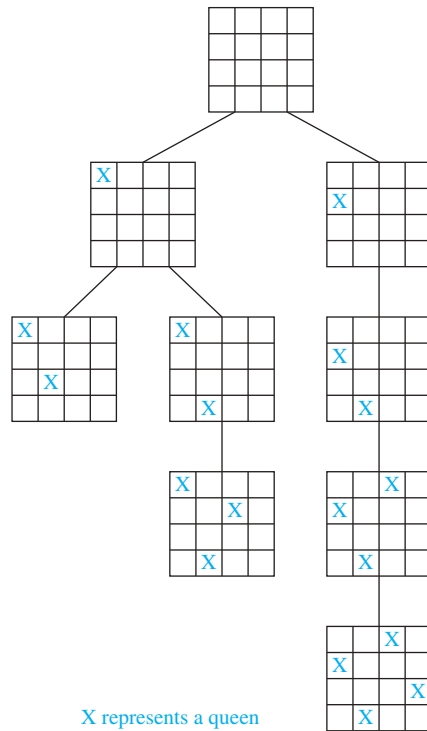
Links

**Solution:** To solve this problem we must find  $n$  positions on an  $n \times n$  chessboard so that no two of these positions are in the same row, same column, or in the same diagonal [a diagonal consists of all positions  $(i, j)$  with  $i + j = m$  for some  $m$ , or  $i - j = m$  for some  $m$ ]. We will use backtracking to solve the  $n$ -queens problem. We start with an empty chessboard. At stage  $k + 1$  we attempt putting an additional queen on the board in the  $(k + 1)$ st column, where there are already queens in the first  $k$  columns. We examine squares in the  $(k + 1)$ st column starting with the square in the first row, looking for a position to place this queen so that it is not in the same row or on the same diagonal as a queen already on the board. (We already know it is not in the same column.) If it is impossible to find a position to place the queen in the  $(k + 1)$ st column, backtrack to the placement of the queen in the  $k$ th column, and place this queen in the next allowable row in this column, if such a row exists. If no such row exists, backtrack further.

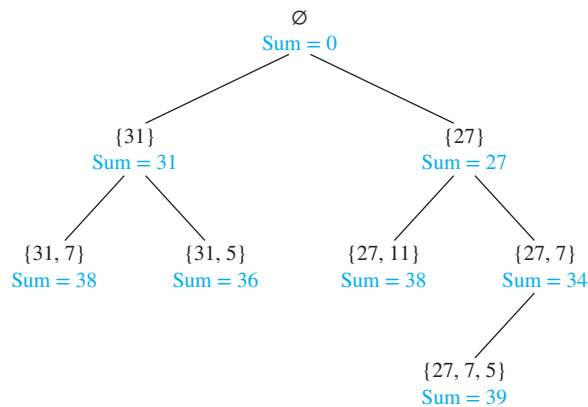
In particular, Figure 12 displays a backtracking solution to the four-queens problem. In this solution, we place a queen in the first row and column. Then we put a queen in the third row of the second column. However, this makes it impossible to place a queen in the third column. So we backtrack and put a queen in the fourth row of the second column. When we do this, we can place a queen in the second row of the third column. But there is no way to add a queen to the fourth column. This shows that no solution results when a queen is placed in the first row and column. We backtrack to the empty chessboard, and place a queen in the second row of the first column. This leads to a solution as shown in Figure 12.

**EXAMPLE 8 Sums of Subsets** Consider this problem. Given a set of positive integers  $x_1, x_2, \dots, x_n$ , find a subset of this set of integers that has  $M$  as its sum. How can backtracking be used to solve this problem?

**Solution:** We start with a sum with no terms. We build up the sum by successively adding terms. An integer in the sequence is included if the sum remains less than  $M$  when this integer is added



**FIGURE 12** A backtracking solution of the four-queens problem.



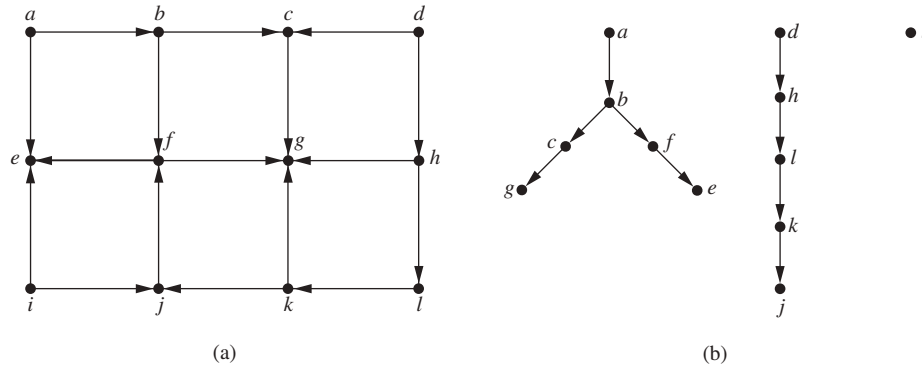
**FIGURE 13** Find a sum equal to 39 using backtracking.

to the sum. If a sum is reached such that the addition of any term is greater than  $M$ , backtrack by dropping the last term of the sum.

Figure 13 displays a backtracking solution to the problem of finding a subset of  $\{31, 27, 15, 11, 7, 5\}$  with the sum equal to 39. 

### 11.4.5 Depth-First Search in Directed Graphs

We can easily modify both depth-first search and breadth-first search so that they can run given a directed graph as input. However, the output will not necessarily be a spanning tree, but rather a spanning forest. In both algorithms we can add an edge only when it is directed away from



**FIGURE 14** Depth-first search of a directed graph.

the vertex that is being visited and to a vertex not yet added. If at a stage of either algorithm we find that no edge exists starting at a vertex already added to one not yet added, the next vertex added by the algorithm becomes the root of a new tree in the spanning forest. This is illustrated in Example 9.

**EXAMPLE 9** What is the output of depth-first search given the graph  $G$  shown in Figure 14(a) as input?

**Solution:** We begin the depth-first search at vertex  $a$  and add vertices  $b$ ,  $c$ , and  $g$  and the corresponding edges where we are blocked. We backtrack to  $c$  but we are still blocked, and then backtrack to  $b$ , where we add vertices  $f$  and  $e$  and the corresponding edges. Backtracking takes us all the way back to  $a$ . We then start a new tree at  $d$  and add vertices  $h$ ,  $l$ ,  $k$ , and  $j$  and the corresponding edges. We backtrack to  $k$ , then  $l$ , then  $h$ , and back to  $d$ . Finally, we start a new tree at  $i$ , completing the depth-first search. The output is shown in Figure 14(b). ◀

Depth-first search in directed graphs is the basis of many algorithms (see [GrYe05], [Ma89], and [CoLeRiSt09]). It can be used to determine whether a directed graph has a circuit, it can be used to carry out a topological sort of a graph, and it can also be used to find the strongly connected components of a directed graph.

We conclude this section with an application of depth-first search and breadth-first search to search engines on the web.

**EXAMPLE 10 Web Crawlers** To index websites, search engines such as Google and Bing systematically explore the web starting at known sites. These search engines use programs called web crawlers (or spiders or bots) to visit websites and analyze their contents. Web crawlers use both depth-first searching and breadth-first searching to create indices. As described in Example 5 in Section 10.1, web pages and links between them can be modeled by a directed graph called the web graph. Web pages are represented by vertices and links are represented by directed edges. When DFS is used to explore the web, an initial web page is selected, a link is followed to a second web page (if there is such a link), a link on the second web page is followed to a third web page, if there is such a link, and so on, until a page with no new links is found. Backtracking is then used to examine links at the previous level to look for new links, and so on. (Because of practical limitations, web crawlers have limits to the depth they search in depth-first search.) Using BFS an initial web page is selected and a link on this page is followed to a second web page, then a second link on the initial page is followed (if it exists), and so on, until all links of the initial page have been followed. Then links on the pages one level down are followed, page by page, and so on.

Search engines have developed sophisticated strategies for web crawlers that take advantage of BFS and DFS. (Google's web crawler is called Googlebot.) They use BFS, starting on high

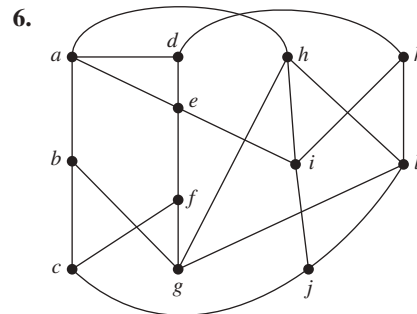
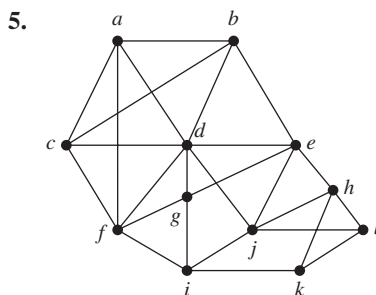
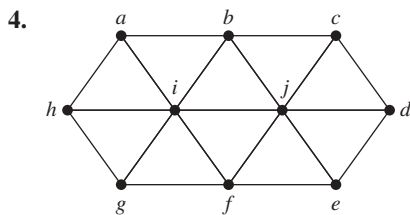
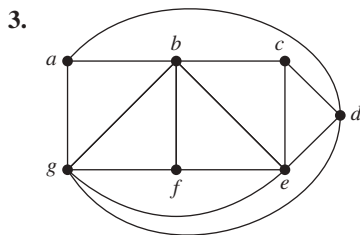
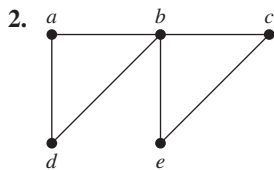
quality web pages, called **seeds**, and explore all links on these pages. As the web crawler proceeds, it adds the urls of all links on the new pages it visits, adding them to the **crawl frontier**. These urls are ordered according to a particular policy, determining the order in which websites are explored.

When seeds have been selected well, Google's approach has been found to quickly reach high quality pages that have many links pointing to them. However, the quality of pages reached decreases as the web crawl continues. This reaches popular web pages well, but it does not reach many other useful, but less popular, web pages. If seed pages do not yield good results, DFS can be used to find candidates for high quality pages, starting at the seeds or other web pages. Also, DFS can be used to reach parts of the web not reached by BFS when it is restricted to a particular number of levels. ▶

## Exercises

- How many edges must be removed from a connected graph with  $n$  vertices and  $m$  edges to produce a spanning tree?

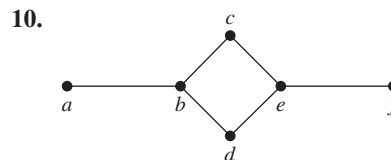
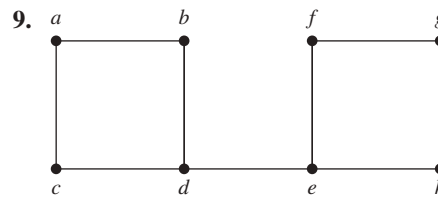
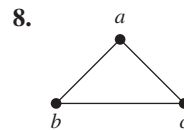
In Exercises 2–6 find a spanning tree for the graph shown by removing edges in simple circuits.



- Find a spanning tree for each of these graphs.

- $K_5$
- $K_{4,4}$
- $K_{1,6}$
- $Q_3$
- $C_5$
- $W_5$

In Exercises 8–10 draw all the spanning trees of the given simple graphs.



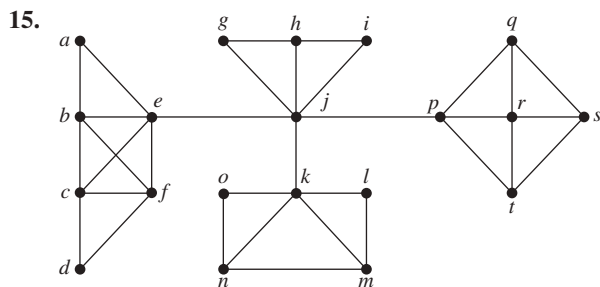
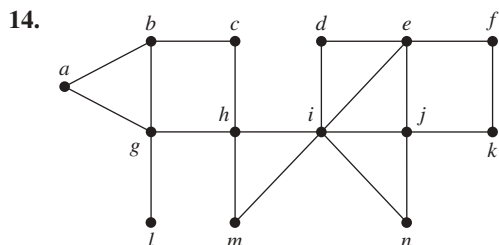
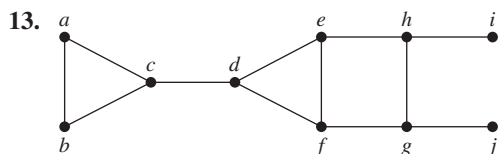
- How many different spanning trees does each of these simple graphs have?

- $K_3$
- $K_4$
- $K_{2,2}$
- $C_5$

- How many nonisomorphic spanning trees does each of these simple graphs have?

- $K_3$
- $K_4$
- $K_5$

In Exercises 13–15 use depth-first search to produce a spanning tree for the given simple graph. Choose  $a$  as the root of this spanning tree and assume that the vertices are ordered alphabetically.



16. Use breadth-first search to produce a spanning tree for each of the simple graphs in Exercises 13–15. Choose  $a$  as the root of each spanning tree.

- 17.** Use depth-first search to find a spanning tree of each of these graphs.

- a)  $W_6$  (see Example 7 of Section 10.2), starting at the vertex of degree 6

- b)  $K_5$**

- c)  $K_{3,4}$ , starting at a vertex of degree 3

- d)  $Q_3$**

- 18.** Use breadth-first search to find a spanning tree of each of the graphs in Exercise 17.

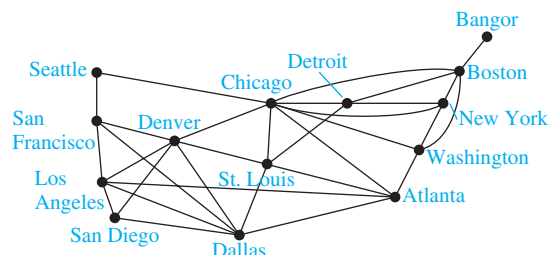
19. Describe the trees produced by breadth-first search and depth-first search of the wheel graph  $W_n$ , starting at the vertex of degree  $n$ , where  $n$  is an integer with  $n \geq 3$ . (See Example 7 of Section 10.2.) Justify your answers.

20. Describe the trees produced by breadth-first search and depth-first search of the complete graph  $K_n$ , where  $n$  is a positive integer. Justify your answers.

21. Describe the trees produced by breadth-first search and depth-first search of the complete bipartite graph  $K_{m,n}$ , starting at a vertex of degree  $m$ , where  $m$  and  $n$  are positive integers. Justify your answers.

22. Describe the tree produced by breadth-first search and depth-first search for the  $n$ -cube graph  $Q_n$ , where  $n$  is a positive integer.

23. Suppose that an airline must reduce its flight schedule to save money. If its original routes are as illustrated here, which flights can be discontinued to retain service between all pairs of cities (where it may be necessary to combine flights to fly from one city to another)?



24. Explain how breadth-first search or depth-first search can be used to order the vertices of a connected graph.

- \*25. Show that the length of the shortest path between vertices  $v$  and  $u$  in a connected simple graph equals the level number of  $u$  in the breadth-first spanning tree of  $G$  with root  $v$ .

- 26.** Use backtracking to try to find a coloring of each of the graphs in Exercises 7–9 of Section 10.8 using three colors.

27. Use backtracking to solve the  $n$ -queens problem for these values of  $n$ .

- a)  $n = 3$**                       **b)  $n = 5$**                       **c)  $n = 6$**

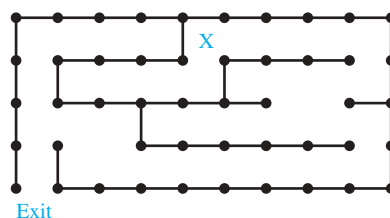
- 28.** Use backtracking to find a subset, if it exists, of the set  $\{27, 24, 19, 14, 11, 8\}$  with sum

- a) 20.                      b) 41.                      c) 60.

- 29.** Explain how backtracking can be used to find a Hamilton path or circuit in a graph.

- 30. a)** Explain how backtracking can be used to find the way out of a maze, given a starting position and the exit position. Consider the maze divided into positions, where at each position the set of available moves includes one to four possibilities (up, down, right, left).

- b) Find a path from the starting position marked by X to the exit in this maze.**



A **spanning forest** of a graph  $G$  is a forest that contains every vertex of  $G$  such that two vertices are in the same tree of the forest when there is a path in  $G$  between these two vertices.

31. Show that every finite simple graph has a spanning forest.
32. How many trees are in the spanning forest of a graph?
33. How many edges must be removed to produce the spanning forest of a graph with  $n$  vertices,  $m$  edges, and  $c$  connected components?
34. Let  $G$  be a connected graph. Show that if  $T$  is a spanning tree of  $G$  constructed using breadth-first search, then an edge of  $G$  not in  $T$  must connect vertices at the same level or at levels that differ by 1 in this spanning tree.
35. Explain how to use breadth-first search to find the length of a shortest path between two vertices in an undirected graph.
36. Devise an algorithm based on breadth-first search that determines whether a graph has a simple circuit, and if so, finds one.
37. Devise an algorithm based on breadth-first search for finding the connected components of a graph.
38. Explain how breadth-first search and how depth-first search can be used to determine whether a graph is bipartite.
39. Which connected simple graphs have exactly one spanning tree?
40. Devise an algorithm for constructing the spanning forest of a graph based on deleting edges that form simple circuits.
41. Devise an algorithm for constructing the spanning forest of a graph based on depth-first searching.
42. Devise an algorithm for constructing the spanning forest of a graph based on breadth-first searching.
43. Let  $G$  be a connected graph. Show that if  $T$  is a spanning tree of  $G$  constructed using depth-first search, then an edge of  $G$  not in  $T$  must be a back edge, that is, it must connect a vertex to one of its ancestors or one of its descendants in  $T$ .
44. When must an edge of a connected simple graph be in every spanning tree for this graph?
45. For which graphs do depth-first search and breadth-first search produce identical spanning trees no matter which vertex is selected as the root of the tree? Justify your answer.
46. Use Exercise 43 to prove that if  $G$  is a connected, simple graph with  $n$  vertices and  $G$  does not contain a simple path of length  $k$  then it contains at most  $(k - 1)n$  edges.
47. Use mathematical induction to prove that breadth-first search visits vertices in order of their level in the resulting spanning tree.
48. Use pseudocode to describe a variation of depth-first search that assigns the integer  $n$  to the  $n$ th vertex visited in the search. Show that this numbering corresponds to the numbering of the vertices created by a preorder traversal of the spanning tree.
49. Use pseudocode to describe a variation of breadth-first search that assigns the integer  $m$  to the  $m$ th vertex visited in the search.

\*50. Suppose that  $G$  is a directed graph and  $T$  is a spanning tree constructed using breadth-first search. Show that every edge of  $G$  has endpoints that are at the same level or one level higher or lower.

51. Show that if  $G$  is a directed graph and  $T$  is a spanning tree constructed using depth-first search, then every edge not in the spanning tree is a **forward edge** connecting an ancestor to a descendant, a **back edge** connecting a descendant to an ancestor, or a **cross edge** connecting a vertex to a vertex in a previously visited subtree.
- \*52. Describe a variation of depth-first search that assigns the smallest available positive integer to a vertex when the algorithm is totally finished with this vertex. Show that in this numbering, each vertex has a larger number than its children and that the children have increasing numbers from left to right.

Let  $T_1$  and  $T_2$  be spanning trees of a graph. The **distance** between  $T_1$  and  $T_2$  is the number of edges in  $T_1$  and  $T_2$  that are not common to  $T_1$  and  $T_2$ .

53. Find the distance between each pair of spanning trees shown in Figures 3(c) and 4 of the graph  $G$  shown in Figure 2.
- \*54. Suppose that  $T_1$ ,  $T_2$ , and  $T_3$  are spanning trees of the simple graph  $G$ . Show that the distance between  $T_1$  and  $T_3$  does not exceed the sum of the distance between  $T_1$  and  $T_2$  and the distance between  $T_2$  and  $T_3$ .
- \*\*55. Suppose that  $T_1$  and  $T_2$  are spanning trees of a simple graph  $G$ . Moreover, suppose that  $e_1$  is an edge in  $T_1$  that is not in  $T_2$ . Show that there is an edge  $e_2$  in  $T_2$  that is not in  $T_1$  such that  $T_1$  remains a spanning tree if  $e_1$  is removed from it and  $e_2$  is added to it, and  $T_2$  remains a spanning tree if  $e_2$  is removed from it and  $e_1$  is added to it.
- \*56. Show that it is possible to find a sequence of spanning trees leading from any spanning tree to any other by successively removing one edge and adding another.

A **rooted spanning tree** of a directed graph is a rooted tree containing edges of the graph such that every vertex of the graph is an endpoint of one of the edges in the tree.

57. For each of the directed graphs in Exercises 18–23 of Section 10.5 either find a rooted spanning tree of the graph or determine that no such tree exists.
- \*58. Show that a connected directed graph in which each vertex has the same in-degree and out-degree has a rooted spanning tree. [Hint: Use an Euler circuit.]
- \*59. Give an algorithm to build a rooted spanning tree for connected directed graphs in which each vertex has the same in-degree and out-degree.
- \*60. Show that if  $G$  is a directed graph and  $T$  is a spanning tree constructed using depth-first search, then  $G$  contains a circuit if and only if  $G$  contains a back edge (see Exercise 51) relative to the spanning tree  $T$ .
- \*61. Use Exercise 60 to construct an algorithm for determining whether a directed graph contains a circuit.